NetShield: Mid-Semester Report

Nov 11, 2026 · @aimal Rextin

1. Introduction and literature survey

An intrusion detector trained on one network should keep working on a network it has never seen, without anyone labelling that new network's traffic first. That is the goal of NetShield, and our course brief sets the rule we follow throughout: **zero labels from the test dataset**. At the midpoint, our current method reaches **96% balanced accuracy, ROC-AUC 0.98 and F1 0.86 on CICIDS2017**, which we kept hidden until a single final run. Earlier versions, which looked at one network flow at a time, were often no better than chance.

Benchmark datasets. We work with five public lab captures: UNSW-NB15 [1], CICIDS2017 and CSE-CIC-IDS2018 [2], ToN-IoT [3] and BoT-IoT [4]. Each mixes recorded normal traffic with attacks run on purpose, and each flow is labelled normal or attack. They differ in network size, recording tool and attack mix, which is exactly why a model trained on one struggles on another.

Shared feature formats. Sarhan et al. re-extracted these captures with one NetFlow tool and proposed a standard feature set, so that results can be compared across datasets [5, 6]. Version 3 of that release adds timing features [7]. We use these NetFlow versions, and our results in Section 3 show that a shared format alone does not make a model transfer.

Why lab results mislead. Sommer and Paxson argued that machine learning works poorly for intrusion detection when normal traffic varies a lot and errors are expensive [8]. Arp et al. list common pitfalls in security ML, including data snooping, spurious correlations and sampling bias [9]. Engelen et al. found labelling and feature-extraction errors in CICIDS2017 [10]. These papers shaped our protocol: duplicates removed, a test set that no choice can see, and the trivial "flag everything" score always reported for comparison.

Domain adaptation. Ben-David et al. show that error on a new domain is bounded by source error plus a distance between the two domains, plus a term for how differently they label data [11]. Alignment methods shrink the distance: CORAL matches feature covariances [12], and DANN learns features a domain classifier cannot tell apart [13]. BBSE corrects for a change in class balance [14]. None of these can fix a mismatch in what counts as an attack, which is the third term. Invariant Risk Minimization looks for features whose link to the label holds in every training environment [15]. That idea led to our consistent-feature filter.

Detecting attacks by host behaviour. Jung et al. detect port scans from the pattern of a host's failed connections over time [16], and BotMiner groups hosts by how they communicate [17]. This view, what a computer does over a window rather than what one flow looks like, is the core of our final method.

Learning without target labels. Isolation Forest finds anomalies with no labels at all [18]. Self-training, or pseudo-labelling, retrains a model on its own confident predictions for unlabelled data [19]. We use both. Our classifiers are LightGBM [20] and Random Forest [21], and our attack cut-off comes from Otsu's method [22], which needs no labels.

The gap we target. Most published detectors are trained and tested on one dataset and score close to 99%. Far fewer test on a different network with no labels from it, and fewer still keep that test hidden until the method is locked. Our project measures exactly that case.

2. Methodology

We train on labelled lab networks and test on a different network whose labels are used only to mark the answers. Every step that touches the test network uses its traffic, never its labels.

2.1 Data

Dataset	Recording tool	Flows in file	Flows used	Attack share in file	Role
UNSW-NB15 (NetFlow v3)	nProbe	2,365,424	1,040,945	5.4%	Training and development
ToN-IoT (NetFlow v3)	nProbe	27,520,260	496,129	39.0%	Training and development
CSE-CIC-IDS2018 (NetFlow v3)	nProbe	20,115,529	1,464,380	12.9%	Training and development
BoT-IoT (NetFlow v3)	nProbe	16,933,808	471,592	99.7%	Training and development
CICIDS2017	CICFlowMeter	2,830,743	2,522,592	19.7%	Hidden test

We removed exact duplicate flows, then sampled up to 3 million flows from the largest files. For BoT-IoT we kept every normal flow, because only 0.3% of it is normal traffic. CICIDS2017 was recorded with a different tool, so its flow columns do not match the others. Our features avoid that problem (2.3).

2.2 Evaluation protocol

1. **Split by clock hour.** In every dataset, odd hours form an unlabelled calibration pool and even hours are the scored test rows. The calibration pool lets the model adapt to the new network without labels.
2. **Develop on four datasets.** Each NetFlow dataset is scored after training on the other three, a leave-one-dataset-out test. We compared 80 configurations this way.
3. **Lock, then test once.** The configuration with the best mean balanced accuracy was written to a file. Only then was CICIDS2017 scored, a single time, and the script refuses a second run.

2.3 Features: what each computer does over time

Every feature is built from five fields that any flow record has: source IP, destination IP, destination port, start time, and whether the flow got a reply. So the features mean the same thing whatever tool recorded the traffic. For each flow we describe its source computer, its destination computer and the pair, in 1-, 10- and 60-minute windows:

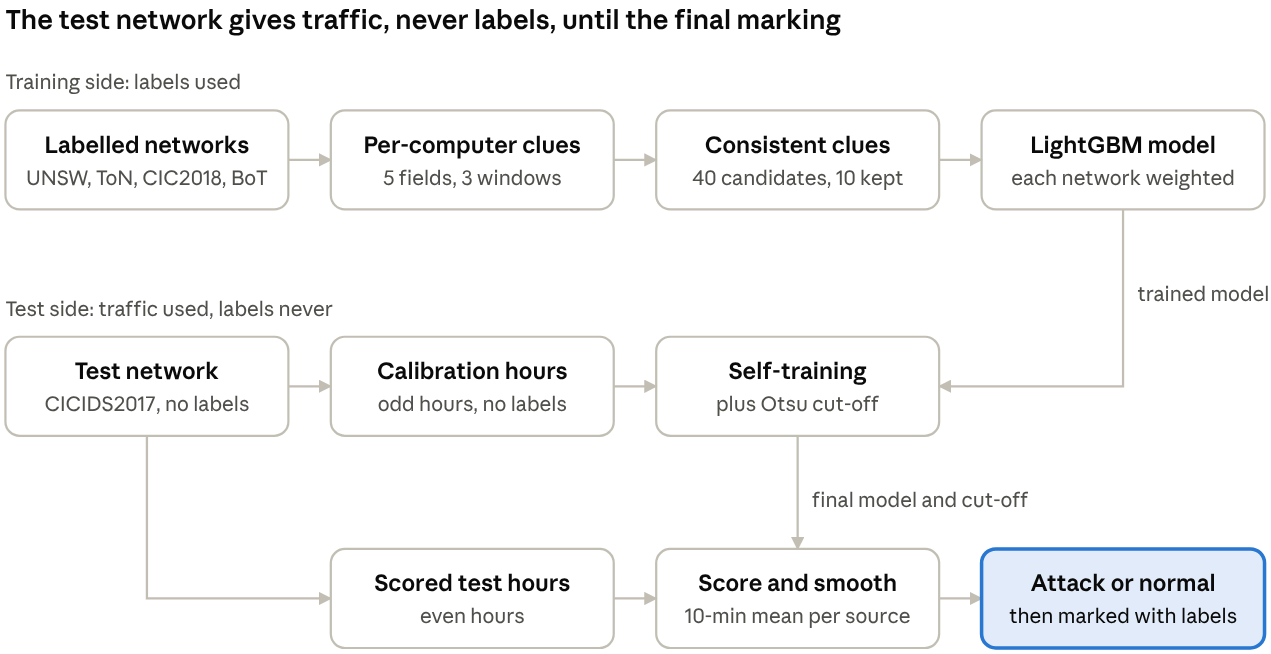
- how many flows, destination computers and ports a source contacted, and how spread out those ports were;
- the share of its flows that got no reply;
- how many times the same port on the same target was hit again;
- how unusual this minute is compared with that computer's own typical minute.

IP addresses are used only for counting and never enter the model. Of 40 candidate features, the filter in 2.4 kept 10.

2.4 Model and zero-label steps

1. **Consistent features only.** We keep a feature only if it points the same way, clearly, in every training network: attacks always higher, or always lower. The cut-off is a single-feature ROC-AUC above 0.55 or below 0.45 in all of them. This drops clues that flip from one network to the next.
2. **LightGBM on several networks.** Each training network counts equally, and inside each one attacks and normal traffic are balanced by sample weights.
3. **Self-training.** The model scores the test network's unlabelled calibration hours. It then retrains on the flows it is most sure about on each side of its cut-off, together with the labelled training data.

4. **Computer smoothing.** Each flow's score is averaged with the mean score of its source computer in the same 10-minute window, because attacks come in bursts from the same machine.
5. **Label-free cut-off.** Otsu's method splits the calibration scores into two groups. Scores above the split are called attacks.



NetShield version 2 · training side and test side

The test network feeds traffic into self-training and the cut-off, and its labels appear only after the final decision.

2.5 Why these choices

Choice	Course concept	Reason
Leave-one-dataset-out development, one-shot hidden test	Model selection, cross-validation, data leakage	Choosing among 80 configurations inflates scores on the data used to choose. A test nobody looked at gives the honest number.

Choice	Course concept	Reason
Equal weight per network and per class	Class imbalance, cost-sensitive learning	Attack shares range from 5% to 99.7%. Unweighted training would learn each network's attack rate, not the attack.
Gradient-boosted trees (LightGBM)	Ensemble learning, bias and variance	Handles skewed counts and missing values with no scaling, and trains in seconds. Random Forest was used in earlier phases.
Consistent-feature filter	Feature selection, spurious correlation, invariance	Our error analysis found clues whose direction flips between networks. Removing them is a direct fix.
Self-training on unlabelled test traffic	Semi-supervised learning	Uses the new network's traffic, which we are allowed to see, without its labels.
Otsu threshold	Unsupervised decision rules	The usual 0.5 cut-off does not carry over between networks, and we cannot tune it with labels.
Balanced accuracy, ROC-AUC, PR-AUC and F1 beside a flag-all baseline	Evaluation under class imbalance	Plain accuracy rewards always saying "attack" when attacks are the majority, as in BoT-IoT.

We also tested Isolation Forest fitted on the test network's own traffic, alone and averaged with the model. It ranked lower on development data, so it was not selected.

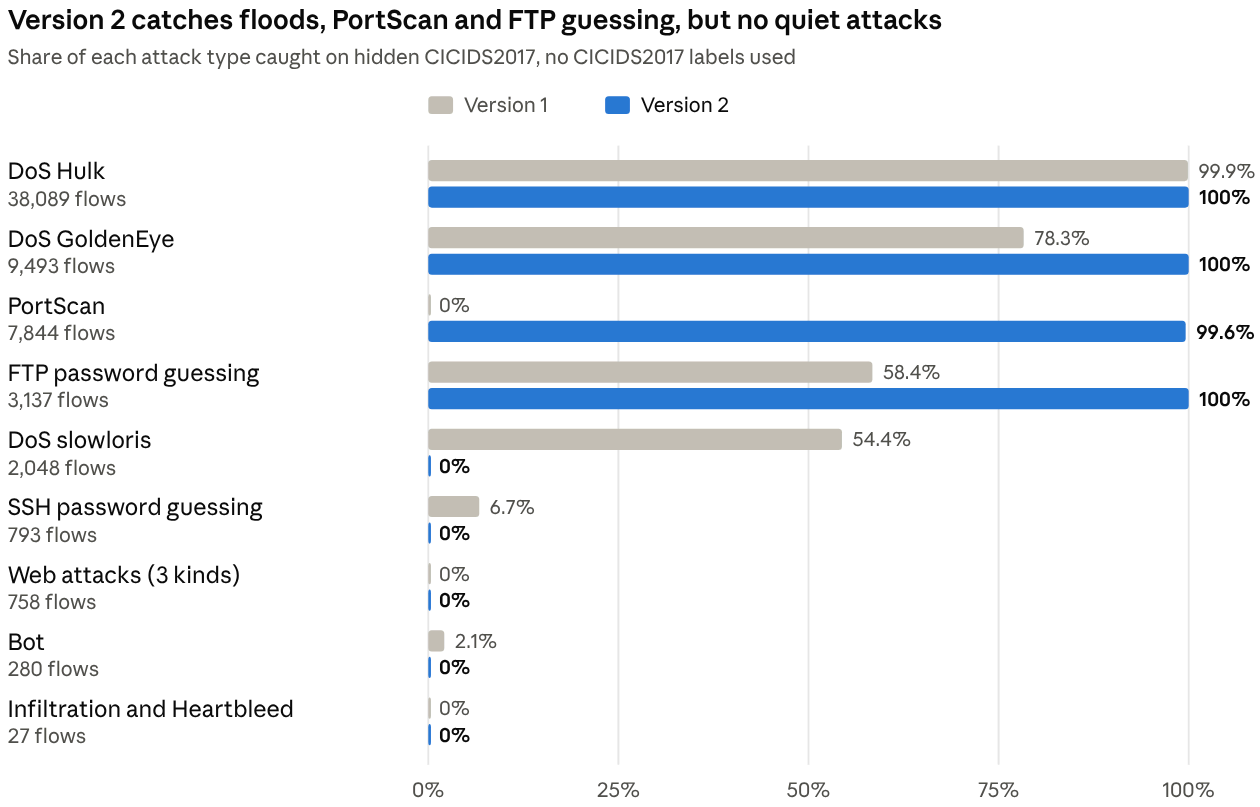
3. Preliminary results

On hidden CICIDS2017, with no CICIDS2017 labels, version 2 caught 94% of attacks with 1.7% false alarms. That is 96% balanced accuracy, against 82% for version 1 on the same test.

3.1 Hidden test: CICIDS2017 (1,000,000 scored flows, 6.2% attacks)

Method	Balanced accuracy	Accuracy	ROC-AUC	PR-AUC	F1	Attacks caught	False alarms
Flag every flow as an attack	0.50	0.06	0.50	0.06	0.12	100%	100%
Version 1: per-computer, 1-minute window	0.82	0.85	0.92	0.72	0.39	78%	14.5%
Version 2: final method	**0.96**	**0.98**	**0.98**	**0.95**	**0.86**	**94%**	**1.7%**

When version 2 flags a flow, it is right about 78% of the time. F1 is lower than accuracy because only 6 in 100 scored flows are attacks, so even a few false alarms weigh heavily next to them.



outputs/zero_label_v2/hidden_by_family.csv · attack types in the scored hours; web attacks, infiltration and Heartbleed rows combined

Version 2 gained PortScan and FTP password guessing but lost slowloris. Every quiet attack type sits at 0%.

3.2 How the score moved during the project

Stage	Test network	Result with no test labels
Original pipeline, one flow at a time	UNSW-NB15 to CICIDS2017	ROC-AUC 0.41, F1 0.28, worse than flagging everything (F1 0.36)
Same NetFlow features on every network	Six directions between UNSW, ToN and CIC2018	Five of six at or below chance (ROC-AUC 0.20 to 0.49)
Version 1: per-computer behaviour	BoT-IoT, hidden	ROC-AUC 0.97, 93% caught, 4.7% false alarms
Version 2: final method	CICIDS2017, hidden	ROC-AUC 0.98, 94% caught, 1.7% false alarms

Each row used a different test, so the rows show direction of travel, not a strict comparison. The last two rows are the only ones scored on data that no earlier choice had seen.

3.3 Development scores

On the four development networks, each scored after training on the other three, version 2's locked configuration averaged 0.89 balanced accuracy, against 0.76 for version 1's settings. UNSW rose most, from 0.47 to 0.85. ToN stayed weakest at 0.80, with 35% false alarms. These numbers are optimistic, because version 2 was the best of 80 configurations on these same datasets. Trained and tested on the same network, the same kind of model scores ROC-AUC 0.97 to 1.00, so the remaining gap comes from moving between networks.

4. Challenges and how we handled them

The biggest obstacle was not the model but the data: the two original datasets measured traffic differently, and some of our early scores were inflated by how they were tested.

4.1 Data issues

Problem	What we saw	What we did
Byte counts meant different things	CICIDS2017 counts payload only, UNSW-NB15 includes headers. The most common CICIDS2017 payload per packet was 0 bytes, while UNSW's minimum was about 40.	Added header bytes back. Development ROC-AUC rose by 0.02 to 0.17 in all ten comparisons.
Duplicates and random splits	337,154 duplicate CICIDS2017 rows, and random train/test splits put the same flow on both sides.	Removed duplicates and split by time.
Unrepresentative attack share	The official UNSW files are 64% attack, the full capture 12.6%. Flagging everything scored F1 0.81.	Report the flag-everything score beside every F1.
Block-sorted test file	The official UNSW test file is sorted by type, so its two halves had 25% and 42% attacks.	Score the whole file instead of halves.
Infinite values	UNSW's NetFlow file stores the byte rate of zero-length flows as infinity. The other files never do.	Treat those values as missing.
Missing columns and clock format	Our CICIDS2017 copy had no IP addresses or times. In the version with them, afternoon times have no AM or PM.	Used the official labelled-flows release, which matched our copy at 2,830,743 rows. Hours before 8 were moved forward 12 hours, giving working hours 8 to 17.

Problem	What we saw	What we did
Download limits	The official NetFlow server needs a sign-in, and the corrected CICIDS2017 download ran at about 18 KB/s.	Used public copies and checked their row and attack counts against the official pages. Corrected CICIDS2017 is still untested.

4.2 Modelling difficulties

- **Domain adaptation broke on raw features.** CORAL on unscaled features flagged 100% of traffic as attacks, and DANN without a log transform did worse than chance. After fixing scaling, both still failed to transfer.
- **Development scores did not hold.** One method fell from ROC-AUC 0.88 on development data to 0.41 on final data. Its settings had been chosen with labels from another part of the test dataset. We switched to choosing on other datasets only, plus a hidden test scored once.
- **A shared feature format was not enough.** With the same 53 NetFlow features everywhere, five of six train and test directions stayed at or below chance, although each network scored ROC-AUC 0.97 to 1.00 on itself.

4.3 Error analysis

- **Clues that flip.** Measured alone, only 3 of 38 per-flow features pointed the same way on all four networks, and none of them was clearly useful on all four. Even per-computer connection counts flipped: attackers were busier than normal computers on three networks but less busy on UNSW, which has only 40 computers. This led to the consistent-feature filter.
- **Scans looked normal.** Version 1 caught 7% of reconnaissance on BoT-IoT. The training attacks were mostly floods and password guessing, which hit few ports, so the model learned "few ports means attack", the opposite of a scan. Longer windows and port-spread features raised PortScan on CICIDS2017 from 0% to 99.6%.
- **Quiet attacks are still missed.** Version 2 caught none of the slow DoS (slowloris), SSH password guessing, web attacks, Bot or infiltration flows in the test hours. Version 1 caught 54% of slowloris, so one of version 2′s changes cost us there. We have not yet isolated which. These attacks look like ordinary sessions at the per-computer level.
- **The headline leans on one attack.** DoS Hulk is 61% of the scored attacks, and both versions caught all of it. The per-attack chart in Section 3 shows the full picture.

5. Revisions to the original plan

We kept the goal and the zero-label rule, but changed the data, the features and the way we test. Each change came from a result that did not hold up.

Original plan	Revision	Reason
UNSW-NB15 and CICIDS2017 only	Added ToN-IoT, CSE-CIC-IDS2018 and BoT-IoT in NetFlow format	With two datasets, every choice is tuned on the test pair. More networks give honest development data and a test that stays hidden.
One shared set of 11 per-flow features	Per-computer behaviour from five basic fields, in three time windows	Per-flow clues flipped direction between networks (4.3). Behaviour such as many ports in a minute means the same thing everywhere.
Align the two datasets with CORAL and DANN, correct class balance with BBSE	Multi-network training, a consistent-feature filter, self-training and computer smoothing	Alignment makes the data look alike but cannot fix a different meaning of "attack" [11].
Choose settings on a development part of the test dataset	Choose on other datasets, then score a hidden dataset once	Using any test-dataset labels to choose breaks the zero-label rule, and the scores did not hold (4.2).

Next steps before the final report

1. **A fresh hidden test.** BoT-IoT and CICIDS2017 have each been used once as the hidden test. We plan to test on the corrected CICIDS2017 release [10] or another public capture that no step has touched.
2. **Quiet attacks.** Slow DoS keeps connections open for a long time, and password guessing repeats short logins that fail. We will test per-computer features for those two patterns and report each attack type separately.
3. **Original pair as a secondary result.** Train on UNSW-NB15 alone with the new features and score CICIDS2017, clearly marked as a second look at that dataset.

References

1. N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," Military Communications and Information Systems Conference (MilCIS), 2015.
2. I. Sharafaldin, A. Habibi Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," ICISSP, 2018.
3. N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," Sustainable Cities and Society, vol. 72, 2021.
4. N. Koroniotis, N. Moustafa, E. Sitnikova and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," Future Generation Computer Systems, vol. 100, 2019.
5. M. Sarhan, S. Layeghy, N. Moustafa and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," Big Data Technologies and Applications, Springer, 2021. doi
6. M. Sarhan, S. Layeghy and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," Mobile Networks and Applications, 2022. doi
7. M. Luay, S. Layeghy, N. Noorbin, M. Sarhan, G. Kulatilleke, N. Moustafa and M. Portmann, "Time matters: temporal NetFlow features for ML-based network intrusion detection," IEEE Access, vol. 14, 2026. doi
8. R. Sommer and V. Paxson, "Outside the closed world: on using machine learning for network intrusion detection," IEEE Symposium on Security and Privacy, 2010.
9. D. Arp et al., "Dos and don'ts of machine learning in computer security," USENIX Security Symposium, 2022.
10. G. Engelen, V. Rimmer and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," IEEE Security and Privacy Workshops, 2021.
11. S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira and J. W. Vaughan, "A theory of learning from different domains," Machine Learning, vol. 79, 2010.
12. B. Sun, J. Feng and K. Saenko, "Return of frustratingly easy domain adaptation," AAAI, 2016.
13. Y. Ganin et al., "Domain-adversarial training of neural networks," Journal of Machine Learning Research, vol. 17, 2016.
14. Z. Lipton, Y.-X. Wang and A. Smola, "Detecting and correcting for label shift with black box predictors," ICML, 2018.

15. M. Arjovsky, L. Bottou, I. Gulrajani and D. Lopez-Paz, "Invariant risk minimization," arXiv:1907.02893, 2019.
16. J. Jung, V. Paxson, A. W. Berger and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," IEEE Symposium on Security and Privacy, 2004.
17. G. Gu, R. Perdisci, J. Zhang and W. Lee, "BotMiner: clustering analysis of network traffic for protocol- and structure-independent botnet detection," USENIX Security Symposium, 2008.
18. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," IEEE International Conference on Data Mining, 2008.
19. D.-H. Lee, "Pseudo-label: the simple and efficient semi-supervised learning method for deep neural networks," ICML Workshop on Challenges in Representation Learning, 2013.
20. G. Ke et al., "LightGBM: a highly efficient gradient boosting decision tree," NeurIPS, 2017.
21. L. Breiman, "Random forests," Machine Learning, vol. 45, 2001.
22. N. Otsu, "A threshold selection method from gray-level histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, 1979.